Sri Lanka Institute of Information Technology



Project Title: **Olist E-Commerce DWBI Solution**
Group: 2026 – DS - 06

Student ID	Name
IT24102981	Jayatunga N.P.
IT24102753	Indrajith A.G.T.
IT24102862	Edirisooriya E.A.S.K.
IT24102941	Rukshan R.A.R.

Data Warehousing and Business Intelligence | IT3101
B.Sc. (Hons) in Information Technology
Data Science

Content

1. Dataset Selection and Business Scenario Identification……..……..…… 1
1.1. Business Domain ……..……...…….. 1
1.2. Purpose of the System ……..……...……... 1
1.3. Business Problem Addressed ……...…….……..… 1
1.4. Dataset Overview …….…….……..… 1
1.5. Attributes Description…….……..1
1.6. Suitability for Developing a DWBI Solution……...…….2

2. Data Source Identification and Preparation ..…… 3
2.1. Description of Data Sources, Formats, and Attributes …….…….. 3
2.2. Relationship Between Sources ……...…… 4
2.3. Data Preparation Steps …….…….…….… 4
Figure 2.3.1. Evidence of Operational Data Preparation…….5
Figure 2.3.1. Evidence of Operational Data Preparation…….5
2.4. Integration into a Unified Analytical Environment …….…….… 6

3. Data Warehouse Architecture Design …….……... 7
3.1. Data Source Layer …….…….. 7
Figure 3.3.1. Data Warehouse Architecture Diagram ……7
3.2. Data Integration Layer (ETL Process) …….....…… 8
3.3. Storage Layer …….……...……..… 8
3.4. Presentation Layer ……..…….8

4. Dimensional Data Warehouse Design and Implementation ……..….... 9
4.1. Business Process and Grain …….9
4.2. Fact Table Design …….……9
4.3. Dimension Table Design ……..……9
4.4. Design Assumption ……...……10
4.5. Star Schema Diagram …….……11

5. ETL Process Development ……..……..12
5.1. Extraction Process …….……12
5.2. Transformation Logic ……12
5.3. Loading Sequence and Validation ……..……13
5.4. Evidence of ETL Implementation ……..13
Figure 5.4.1. Extraction Phase ……..……13
Figure 5.4.2. Transformation Phase ……...…14
Figure 5.4.3. Loading Phase ……...…..15

6. Data Mart Development ……16
6.1. Overview and Architecture of Data Marts ……..……..16
6.2. Department-Specific Data Mart 1: Customer Experience & Logistics …….16
Figure 6.2.1. SQL Script of Data Mart 1 ……..….16
Figure 6.2.2. Evidence of the Data Mart 1 Creation ……....17
6.3. Department-Specific Data Mart 2: Sales and Revenue Performance ……....17
Figure 6.3.1. SQL Script of Data Mart 2 ……..….18
Figure 6.3.2. Evidence of the Data Mart Creation ……...18

7. OLAP Analysis and Business Intelligence Dashboard Development ……...…19
7.1. Visualization Strategy ……19
7.2. DAX Measures Implementation ……19
7.3. Dashboard Structure and Implementation ……..…19
7.4 Evidence of Interactive Dashboards ……...21
Figure 7.4.1. Page 1 ……..21
Figure 7.4.2. Page 2 ……..……21
Figure 7.4.3. Page 2 ……..22
Figure 7.4.4. Page 3 ……..……22
Figure 7.4.5. Page 3 ……..……23
Figure 7.5.6. Page 3 ……..…23

8. Business Insights and Recommendations ……...……..24
9. AI Acknowledgement ……...…….26

Task 1: Dataset Selection and Business Scenario Identification

1.1 Business Domain

The selected domain for this project is Retail / E-commerce.

1.2 Purpose of the System

Olist is a famous e-commerce platform in Brazil that enables small and medium enterprises (SMEs) to sell their products to consumers. Olist's operational system helps with all the aspects of the shopping process, including looking up inventory and checking out through one's shopping cart.

1.3 Business Problem Addressed

At present, Olist's operational data is evenly distributed and divided among different systems. The team in charge of the business has trouble gaining complete and accurate business intelligence on delays in delivery, rising transport costs, and the reasons behind low customer satisfaction. The need for running complicated and deep analytical queries has turned the OLTP database into an extremely inefficient tool that hampers the business's ability to execute any supply-chain optimizations.

1.4 Dataset Overview

- **Dataset Source:** The Olist Brazilian E-Commerce Public Dataset, accessed via Kaggle. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
- **Dataset Format:** Originally provided as a collection of 9 relational CSV files.
- **Number of Records:** The dataset contains approximately 100,000 distinct operational e-commerce orders placed between 2016 and 2018, encompassing over 112,000 individual order items.

1.5 Attributes Description

The dataset contains multiple attributes suitable for dimensional analysis, including:

- **Transactional Attributes:** order_id, price, freight_value, payment_installments.
- **Temporal Attributes:** order_purchase_timestamp, order_delivered_customer_date.
- **Categorical/Descriptive Attributes:** product_category_name, customer_city, customer_state, review_score.

1.6 Suitability for Developing a DWBI Solution

This dataset is highly suitable for this assignment because it fulfills all fundamental dimensional modeling requirements:

- **Represents an OLTP System:** It accurately mirrors a highly normalized operational database with primary and foreign key constraints (e.g., separating orders, payments, and items).
- **Supports Fact and Dimension Creation:** The transactional line items (olist_order_items) naturally form a Fact Table containing numerical measures (price, freight), while customers, products, and locations serve as rich descriptive Dimension Tables.
- **Sufficient Volume:** With over 100k records, it provides enough data depth to demonstrate meaningful aggregations, drill-downs, and trend analysis in the final BI dashboard.

Task 2: Data Source Identification and Preparation

In order to meet the need for using a variety of sources and to create an actual architecture environment, the data was logically separated into various environments.

2.1 Description of Data Sources, Formats, and Attributes

To provide a comprehensive analytical view, all nine entities of the operational dataset were categorized into three distinct source systems.

Operational Source System	Source Format	Files / Tables Utilized	Key Available Attributes
1. Core E-Commerce OLTP System:	Relational Database (SQL Server)	1. olist_orders 2. olist_order_items 3. olist_order_payments 4. olist_products	order_id, customer_id, order_status, order_purchase_timestamp, product_id, price, freight_value, payment_type, payment_installments, product_weight_g
2. CRM, Support & Logistics Portal:	Flat Files (CSV)	5. olist_customers.csv 6. olist_sellers.csv 7. olist_order_reviews.csv 8. olist_geolocation.csv	customer_id, customer_unique_id, seller_id, customer_city, review_score, review_comment_message, geolocation_zip_code_prefix, geolocation_lat, geolocation_lng
3. Product Translation Microservice:	JSON File	9. category_catalog_api.json	product_category_name, product_category_name_english

2.2 Relationship Between Sources

While originating from different forms and dissociated systems, these separate sources share logical connections that will be established during the ETL connection stage.

- **Order to Customer (SQL to CSV**): The OLTP olist_orders table connects to the flat-file olist_customers.csv via the shared customer_id business key.
- **Item to Seller (SQL to CSV):** The OLTP olist_order_items table connects to the flat-file olist_sellers.csv via the seller_id key.
- **Transactions to Feedback (SQL to CSV):** The core olist_orders database table links to the olist_order_reviews.csv export via the order_id, allowing the business to correlate delivery delays with 1-star ratings.
- **Demographics to Geography (CSV to CSV):** Both olist_customers.csv and olist_sellers.csv link to the olist_geolocation.csv reference file using the zip_code_prefix to map exact latitude/longitude coordinates.
- **Products to Translations (SQL to JSON):** The OLTP olist_products table maps to the category_catalog_api.json file via the Portuguese product_category_name attribute, enabling English standardisation.

2.3 Data Preparation Steps

Prior to initiating the core ETL pipeline, the following preparation steps were executed to set up the operational environment:

1. **Relational Database Provisioning:** A SQL Server database named Olist_OLTP_Live was created to host the live transactional tables.
2. **Schema and Constraint Adjustments**: During the flat-file import into SQL Server, columns representing legitimate operational absences, such as order_delivered_customer_date (for pending orders) and product_weight_g (for unmeasured items)were explicitly configured to ALLOW NULLS to prevent DBNull ingestion errors.
3. **Data Type Correction**: The product_weight_g column was modified from a smallint to an int data type to accommodate oversized freight values exceeding 32,767, successfully preventing data overflow exceptions.
4. **Format Conversion:** The product category reference data was parsed using a Python script (Pandas) and exported as a structured JSON array to simulate a decoupled REST API response.

Evidence of Operational Data Preparation:

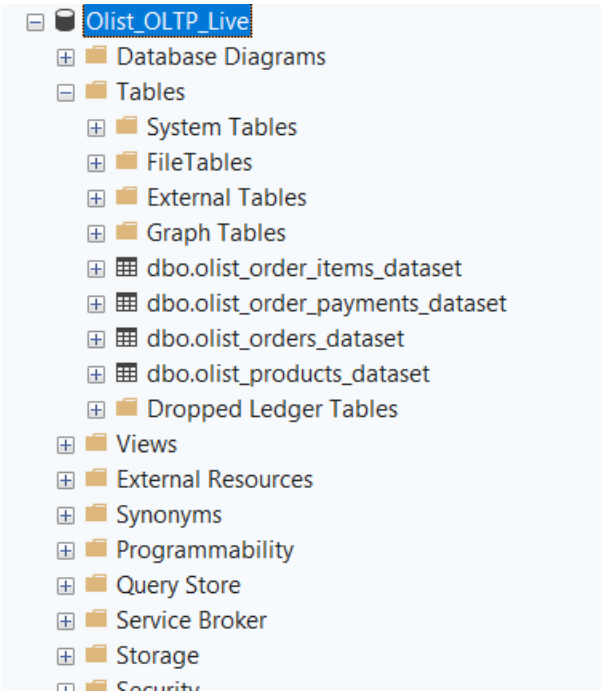


Figure 2.3.1 : SSMS showing the Olist_OLTP_Live database and the 4 imported relational tables

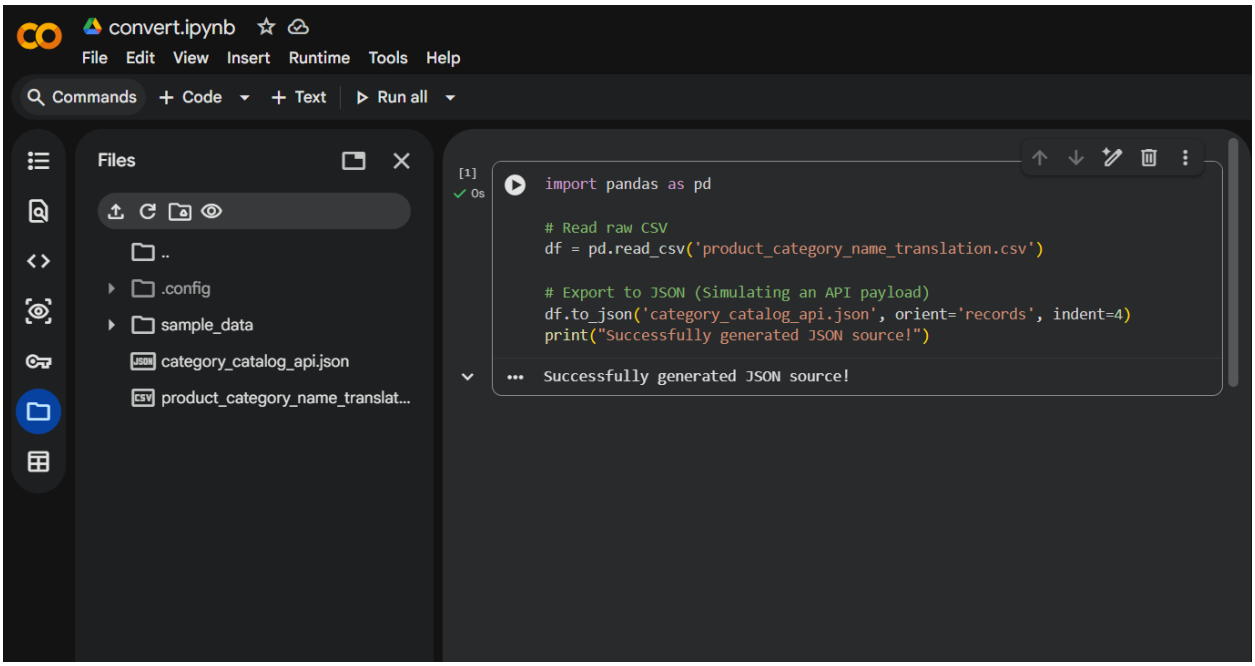


Figure 2.3.2 : Google Colab Python script demonstrating the JSON file generation

2.4 Integration into a Unified Analytical Environment

Combining data from multiple operational systems into one analytical framework involves separating tasks of reading data for analysis from writing data for day-to-day transactions. SQL databases and APIs are types of operational systems whose designs do not favor interaction with each other in providing the ability to do large-scale historical reporting. One of the main stages of the integration process is to implement an ETL pipeline. The ETL system is able to connect to a SQL server, CSV file, and JSON file at the same time and perform the extraction process of the data. Then, during Transformation some of the structural clash is resolved that may happen, for example the standardizing of date formats (text-based) from the CSV file into the native date formats. Furthermore, there is a matching between English text-based identifiers from the JSON file and the SQL product identifiers in data warehouse system. Finally, the data transformed is placed in the Data Warehouse so that one forms an integrated and cleaned "Single source of the Truth" that can be queried by BI software for the analysis across the departments but is not interrupting the process of conducting business.

Task 3: Data Warehouse Architecture Design

To build a robust and scalable Business Intelligence solution for Olist, a comprehensive 3-Tier Data Warehouse Architecture has been designed.

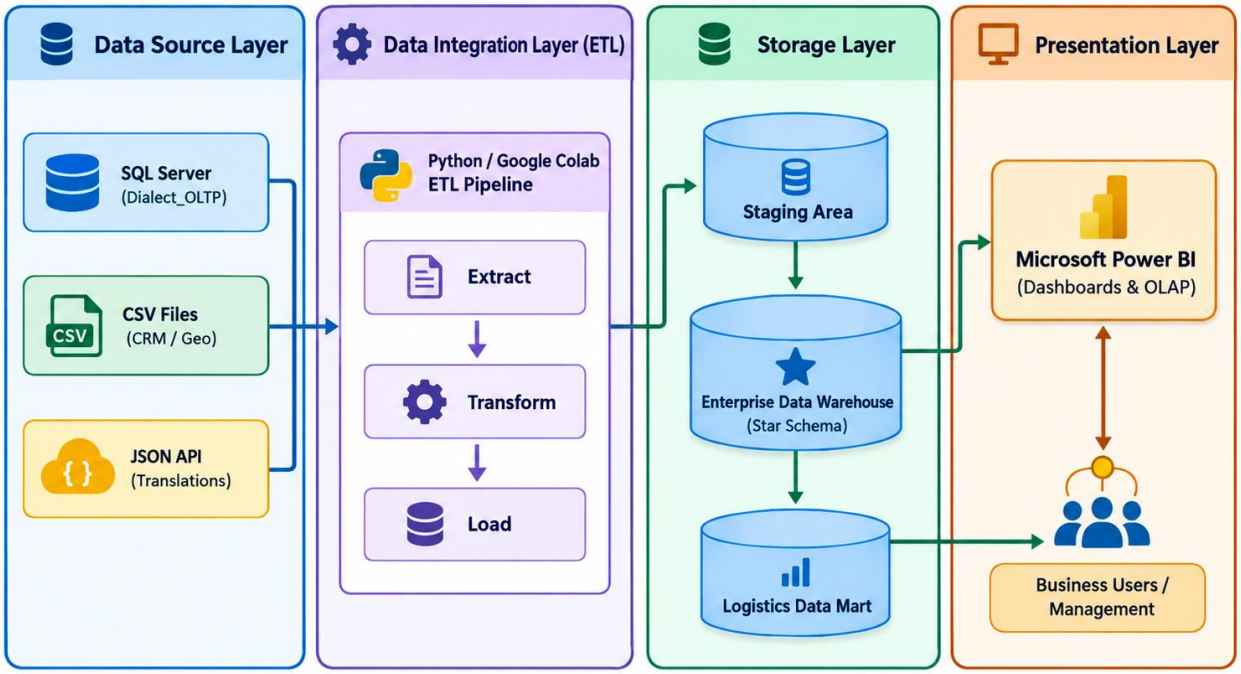


Figure 3.1.1 : High-Level Data Warehouse and Business Intelligence Architecture for Olist

Explanation of Architecture Components:

3.1 Data Source Layer

The layer consists of all the various environments that lead to the generation of raw business data. Specifically, this layer contains:

- **Operational Databases:** Active transactions and inventories are handled through the live MS SQL Server instance being the Olist_OLTP_Live database.
- **External Files:** Any feedback related to CRM in addition to seller information and mapping are handled through legacy CSV flat files.
- **Application Systems:** The external JSON API microservice that is responsible for translating categories of products into English.

3.2 Data Integration Layer (ETL Process)

The identified layer is the one that enables the execution of the function that bridges the operations and storage for analysis. Python has been used through Google Colab and Pandas to come up with three subprocesses as it follows:

- **Extraction:** It runs the connection of SQL, CSV and JSON altogether.
- **Transformation:** It is responsible for cleaning the data to remove missing data, converting data from one type to another, removing duplicates of data and join translation websites to convert product IDs.
- **Loading:** It is about uploading the processed data into the Data Warehouse.

3.3 Storage Layer

The storage layer comprises the centralized storage for past and assimilated data, divided into three functional areas:

- **Staging Area:** A temporary storage area that holds the raw data before processing and provides some protection for current source systems from the effects of data loading.
- **Enterprise Data Warehouse (EDW):** The centralized storage facility created using the dimensional Star Schema. It contains a denormalized Fact_Orders table and relevant dimensions (Time, Customer, Product, Location).
- **Data Mart:** A highly specific part of the EDW, focused on one specific department. For this project, two Data Marts will be created: the "Customer Experience & Logistics" data mart to provide special support for the supply chain team in studying delivery delays and customer satisfaction, and the "Sales and Revenue Performance" data mart to assist the finance and sales teams in analyzing revenue trends and product profitability.

3.4 Presentation Layer

The top most layer is what users see and takes in data.

- **Tools for Reporting and the Dashboard of BI:** The power of Microsoft Power BI is put to use in such a way that it can establish a direct link with the EDW and Data Mart.
- **OLAP analysis:** The presentation layer can perform online analytical processing (OLAP). As a result, it helps users of the business perform drill-downs, roll-ups, and timelines analysis through the use of visual and slicer.

Task 4: Dimensional Data Warehouse Design and Implementation

In order to facilitate effective OLAP and dashboard reporting, the operational data was simplified using Star Schema structure.

4.1 Business Process and Grain

- **Process:** The principal process that we are analysing is sales via an ecommerce website and the order processing cycle from a customer placing an order to the order being delivered.
- **Grain:** The grain of the Fact Table is defined at the level of an "Order Line Item." This is the most granular level of data there could be. Each row in the table represents a specific product item ordered. The specific grain is selected since a single order can have a lot of different products with its own price, shipping value, and seller.

4.2 Fact Table Design

- **Table Name:** Fact_Sales
- **Foreign Keys (FK):** Date_Key, Customer_Key, Product_Key, Location_Key, Seller_Key.
- **Measures (Additive & Semi-Additive):**
    - ✓ Item_Price (Revenue from the product)
    - ✓ Freight_Value (Cost of shipping)
    - ✓ Total_Order_Value (Calculated as Price + Freight Value)
    - ✓ Delivery_Duration_Days (Calculated duration between purchase and delivery)
    - ✓ Review_Score (Customer satisfaction rating out of 5, merged from the CRM support portal).
    - ✓ Payment_Type

4.3 Dimension Tables Design

The schema includes the following dimensions, designed with Surrogate Keys (SK) as Primary Keys (PK) to isolate the data warehouse from operational changes.

Dimension Table	Primary Key (SK)	Natural Key (NK)	Descriptive Attributes & Hierarchies
Dim_Customer	Customer_Key	customer_id	customer_unique_id. (Customer behavior profiling).
Dim_Product	Product_Key	product_id	product_category_english, weight_g, length_cm. Hierarchy: *Category → Product*
Dim_Location	Location_Key	zip_code_prefix	city, state, latitude, longitude. Hierarchy: *State → City → Zip Code*
Dim_Seller	Seller_Key	seller_id	seller_city, seller_state.
Dim_Date	Date_Key	full_date	Year, Quarter, Month, Day, DayOfWeek. Hierarchy: *Year → Quarter → Month → Date*

4.4 Design Assumptions

During the dimensional modeling process, the following assumptions were made:

- **Surrogate Keys:** Digital surrogate keys (for example, Customer_Key, Product_Key) were created for the dimension tables with the goal of improving join performance in the BI tool instead of relying on the lengthy IDs found in the source data.
- **Date Dimension:** The order_purchase_timestamp was selected as the main event date which connects the Fact table with the Date Dimension since it reflects the moment sales occur and revenue generation begins.
- **Separated Location Data:** Rather than attaching location data to the Customer and Seller dimensions, a separate Dim_Location was created and used.

4.5 Star Schema Diagram

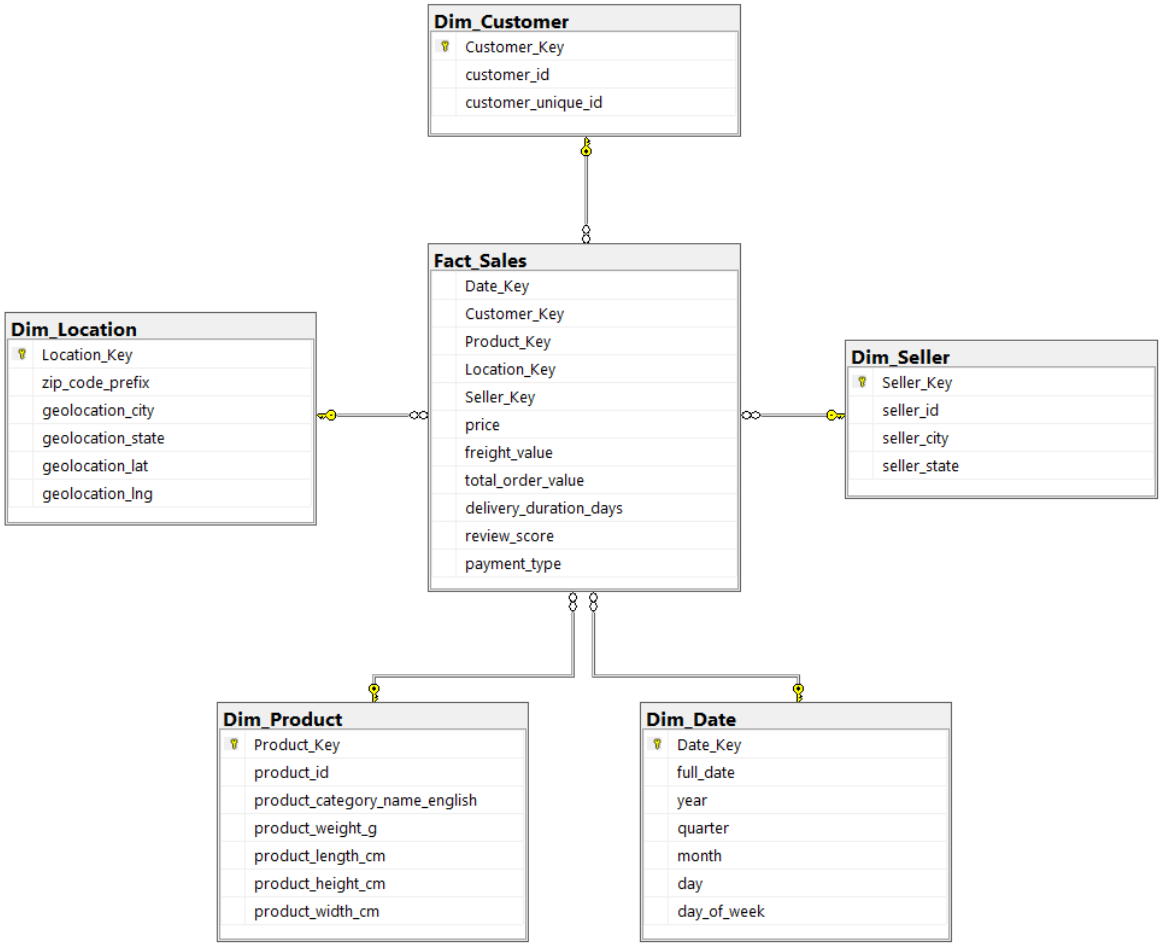


Figure 4.5 : Star Schema Dimensional Model for Olist E-Commerce

Task 5: ETL Process Development

The ETL process, which is short for Extract, Transform and Load, was developed utilizing the Pandas library in Python using Google Colab. This process acts as the integration layer that transfers data from independent operational systems into the centralized Data Warehouse.

5.1 Extraction Process

- **Methods and Connections:** The data was fetched using the conventional I/O connectors (pd.read_csv and pd.read_json).
- **Execution:** At this point, the pipeline has been able to connect to all 9 independent source files at the same time. These are the simulated Relational Database tables (Orders, Items, Products, Payments), the flat files from the CRM (Customers, Sellers, Geolocation, Order Ratings), and the external JSON translation API.

5.2 Transformation Logic

There are numerous transformation rules in place to achieve good data quality and consistency.

1. **Handling of Missing Values:** The column product_weight_g contained operational null values. For the dataset, this value had been filled in with the median weight of the dataset. Where there were units with unknown product categories, the nulls were replaced by the string "Unknown".
2. **Removal and Cleaning of Duplicates:**
   - The original olist_geolocation file contained more than a million rows with duplicate ZIP codes. A GROUP BY operation was used to extract the unique ZIP codes and find their average latitude and longitude before loading it to Dim_Location.
   - The olist_order_reviews file was also containing repeated reviews for several orders. Here, to keep only the latest review score per order, the drop_duplicates function had been used before merging it with the Fact table.
3. **Data Type Conversion:** In the conversion process of text-based time data (e.g., order_purchase_timestamp), custom datetime64 objects were employed to facilitate the process of extracting time-series.
4. **Standardization:** The JSON API file was standardised with the help of the products table to convert all Portuguese product categories to English (product_category_name_english).
5. **Generation of Derived attributes:**The Date table contains attributes such as Year and Quarter and Day_of_Week derived from the purchased timestamp. A new measure called Total Order Value was introduced in Fact_Sales as price + freight_value. The delivery_duration_days was calculated by using an approach that involved subtracting purchase date from the delivery date. Finally, the cleansed review score was brought into the Fact table and the orders not having a review were assigned with an index score of 0.

5.3 Loading Sequence and Validation

- **Loading Sequence:** To maintain referential integrity, the Dimension tables (Dim_Location, Dim_Date, Dim_Product, Dim_Customer, Dim_Seller) were loaded first. The Fact_Sales table was loaded last, as it relies on the Primary Keys of the dimensions.
- **Validation Results:** The pipeline executed successfully without data truncation or overflow errors. The row counts in the staging outputs matched the expected aggregated logical models.

5.4 Evidence of ETL Implementation

```
import pandas as pd
import numpy as np

print("--- Starting ETL Process ---")

# 1. EXTRACTION (All 9 Files)
df_orders = pd.read_csv('olist_orders_dataset.csv')
df_items = pd.read_csv('olist_order_items_dataset.csv')
df_products = pd.read_csv('olist_products_dataset.csv')
df_customers = pd.read_csv('olist_customers_dataset.csv')
df_sellers = pd.read_csv('olist_sellers_dataset.csv')
df_geo = pd.read_csv('olist_geolocation_dataset.csv')
df_translation = pd.read_json('category_catalog_api.json')
df_reviews = pd.read_csv('olist_order_reviews_dataset.csv')
df_payments = pd.read_csv('olist_order_payments_dataset.csv')

print("Extraction Complete.")
```

```
--- Starting ETL Process ---
Extraction Complete.
```

Figure 5.4.1 : Showing the Extraction phase

```
# 2. TRANSFORMATION (Cleaning & Generating Surrogate Keys)

# --- A. Date Cleaning ---
df_orders['order_purchase_timestamp'] = pd.to_datetime(df_orders['order_purchase_timestamp'], errors='coerce')
df_orders['order_delivered_customer_date'] = pd.to_datetime(df_orders['order_delivered_customer_date'], errors='coerce')

# --- B. Dim_Location ---
dim_location = df_geo.groupby('geolocation_zip_code_prefix').agg({
    'geolocation_city': 'first',
    'geolocation_state': 'first',
    'geolocation_lat': 'mean',
    'geolocation_lng': 'mean'
}).reset_index()
dim_location.rename(columns={'geolocation_zip_code_prefix': 'zip_code_prefix'}, inplace=True)
dim_location['Location_Key'] = range(1, len(dim_location) + 1) # Create Surrogate Key

# --- C. Dim_Product ---
df_products['product_weight_g'].fillna(df_products['product_weight_g'].median(), inplace=True)
dim_product = pd.merge(df_products, df_translation, on='product_category_name', how='left')
dim_product = dim_product[['product_id', 'product_category_name_english', 'product_weight_g', 'product_length_cm', 'product_height_cm', 'prc
dim_product['product_category_name_english'].fillna('Unknown', inplace=True)
dim_product['Product_Key'] = range(1, len(dim_product) + 1) # Create Surrogate Key

# --- D. Dim_Customer ---
dim_customer = df_customers[['customer_id', 'customer_unique_id']].drop_duplicates()
dim_customer['Customer_Key'] = range(1, len(dim_customer) + 1) # Create Surrogate Key

# --- E. Dim_Seller ---
dim_seller = df_sellers[['seller_id', 'seller_city', 'seller_state']].drop_duplicates()
dim_seller['Seller_Key'] = range(1, len(dim_seller) + 1) # Create Surrogate Key
```

```
# --- F. Dim_Date ---
dim_date = pd.DataFrame({'full_date': df_orders['order_purchase_timestamp'].dt.date.dropna().unique()})
dim_date['full_date'] = pd.to_datetime(dim_date['full_date'])
dim_date['Date_Key'] = dim_date['full_date'].dt.strftime('%Y%m%d').astype(int)
dim_date['year'] = dim_date['full_date'].dt.year
dim_date['quarter'] = dim_date['full_date'].dt.quarter
dim_date['month'] = dim_date['full_date'].dt.month
dim_date['day'] = dim_date['full_date'].dt.day
dim_date['day_of_week'] = dim_date['full_date'].dt.day_name()

# --- G. Clean Reviews & Payments ---
df_reviews_clean = df_reviews[['order_id', 'review_score']].drop_duplicates(subset=['order_id'])
df_payments_clean = df_payments[['order_id', 'payment_type']].drop_duplicates(subset=['order_id'])

# --- H. Fact Table (Merging Everything using Natural Keys, then keeping Surrogate Keys) ---
fact_sales = pd.merge(df_items, df_orders, on='order_id', how='inner')
fact_sales = pd.merge(fact_sales, dim_customer, on='customer_id', how='left')
fact_sales = pd.merge(fact_sales, dim_product, on='product_id', how='left')
fact_sales = pd.merge(fact_sales, dim_seller, on='seller_id', how='left')

# Map Location_Key using customer's zip code
fact_sales = pd.merge(fact_sales, df_customers[['customer_id', 'customer_zip_code_prefix']], on='customer_id', how='left')
fact_sales = pd.merge(fact_sales, dim_location[['zip_code_prefix', 'Location_Key']], left_on='customer_zip_code_prefix', right_on='zip_code_

# Add Reviews and Payments
fact_sales = pd.merge(fact_sales, df_reviews_clean, on='order_id', how='left')
fact_sales = pd.merge(fact_sales, df_payments_clean, on='order_id', how='left')

fact_sales['review_score'].fillna(0, inplace=True)
fact_sales['payment_type'].fillna('not_defined', inplace=True)

# Generate Date_Key for Fact Table
fact_sales['Date_Key'] = fact_sales['order_purchase_timestamp'].dt.strftime('%Y%m%d').fillna(0).astype(int)

# Derived Measures
fact_sales['total_order_value'] = fact_sales['price'] + fact_sales['freight_value']
fact_sales['delivery_duration_days'] = (fact_sales['order_delivered_customer_date'] - fact_sales['order_purchase_timestamp']).dt.days

# Keep ONLY the Star Schema Columns (Drop all old text IDs!)
fact_sales = fact_sales[['Date_Key', 'Customer_Key', 'Product_Key', 'Location_Key', 'Seller_Key', 'price', 'freight_value', 'total_order_val

# Drop rows where critical foreign keys couldn't be matched
fact_sales.dropna(subset=['Date_Key', 'Customer_Key', 'Product_Key', 'Location_Key', 'Seller_Key'], inplace=True)

# Convert Keys to Integer to prevent decimals in CSV
keys_cols = ['Date_Key', 'Customer_Key', 'Product_Key', 'Location_Key', 'Seller_Key']
fact_sales[keys_cols] = fact_sales[keys_cols].astype(int)

print("Transformation Complete.")
```

```
/tmp/ipykernel_2773/2242154003.py:18: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignmen
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values al

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(


  df_products['product_weight_g'].fillna(df_products['product_weight_g'].median(), inplace=True)
/tmp/ipykernel_2773/2242154003.py:21: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignmen
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values al

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(


  dim_product['product_category_name_english'].fillna('Unknown', inplace=True)
/tmp/ipykernel_2773/2242154003.py:60: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignmen
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values al

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(


  fact_sales['review_score'].fillna(0, inplace=True)
/tmp/ipykernel_2773/2242154003.py:61: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignmen
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values al

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(


  fact_sales['payment_type'].fillna('not_defined', inplace=True)
Transformation Complete.
```

Figure 5.4.2 : Showing the Transformation phase

```
# 3. LOADING (Exporting all 6 clean tables)
dim_location.to_csv('Clean_Dim_Location.csv', index=False)
dim_product.to_csv('Clean_Dim_Product.csv', index=False)
dim_customer.to_csv('Clean_Dim_Customer.csv', index=False)
dim_seller.to_csv('Clean_Dim_Seller.csv', index=False)
dim_date.to_csv('Clean_Dim_Date.csv', index=False)
fact_sales.to_csv('Clean_Fact_Sales.csv', index=False)

print("Data exported successfully! Please download ALL 6 Clean CSV files.")
```

```
Data exported successfully! Please download ALL 6 Clean CSV files.
```

Figure 5.4.3 : Showing the Loading phase

Task 6: Data Mart Development

6.1 Overview and Architecture of Data Marts

EDW, or Enterprise Data Warehouse, employs a Star Schema architecture by connecting data processed throughout the organization, while Data Marts serve as highly specialized access layers created for particular organizational sectors. Two proper Data Marts have been introduced in the Olist E-commerce project via establishing virtual layers as SQL Server views for improving reporting functionality and meeting different requirements of departments.

6.2 Department-Specific Data Mart 1: Customer Experience & Logistics

- **Purpose of the Data Mart:** The purpose of this Data Mart is to assess operational performance through combining the measurements of delivery with the customer feedback. The main aim is to keep checking the effectiveness of fulfillment and study how the logistical obstacles affect the satisfaction of customers in various areas and of different types of products.
- **Target Users:** Customer Support Teams, Logistics Managers, and Operations Quality Assurance personnel.
- **Analytical Benefits:**
  - ✓ Provides an opportunity for cross analysis for the time of delivery (delivery_duration_days) and customer satisfaction (review_score) which makes it possible to detect windows of failed delivery.
  - ✓ Shows the friction of logistics in regions by analyzing the state of customer (customer) and seller.
  - ✓ Helps with the optimization of delivery services agreements and decreasing the time of delivery.

```sql
-- Creating the Customer Experience and Logistics Data Mart
CREATE VIEW DM_Customer_Experience AS
SELECT
    d.year,
    d.month,
    l.geolocation_state AS customer_state,
    p.product_category_name_english AS product_category,
    s.seller_state,
    f.delivery_duration_days,
    f.review_score,
    f.total_order_value,
    f.payment_type
FROM Fact_Sales f
JOIN Dim_Date d ON f.Date_Key = d.Date_Key
JOIN Dim_Location l ON f.Location_Key = l.Location_Key
JOIN Dim_Product p ON f.Product_Key = p.Product_Key
JOIN Dim_Seller s ON f.Seller_Key = s.Seller_Key;
```

Figure 6.2.1 : SQL Script of Data Mart 1

```
98
99  SELECT * FROM DM_Customer_Experience;
```

00 % 10 0

Results Messages

	year	month	customer_state	product_category	seller_state	delivery_duration_days	review_score	total_order_value	payment_type
	2017	9	RJ	health_beauty	SP	10	5	104.37	credit_card
	2018	8	GO	sports_leisure	SP	12	5	48.87	debit_card
	2018	6	PB	auto	SP	19	4	119.7	debit_card
	2017	4	SC	health_beauty	SP	9	5	79.61	credit_card
	2017	12	MG	bed_bath_table	SP	7	5	44.89	debit_card
	2018	6	SP	office_furniture	SP	4	4	377.07	credit_card
	2017	5	CE	watches_gifts	SP	8	5	601.09	credit_card
	2017	6	ES	perfumery	MG	8	4	119.84	credit_card
	2018	2	SP	health_beauty	SP	4	3	70.72	credit_card
0	2018	8	RJ	toys	SP	0	1	121.31	credit_card
1	2018	8	RJ	toys	SP	0	1	121.31	credit_card
2	2018	5	PR	watches_gifts	SP	12	1	199.04	credit_card
3	2018	6	MG	baby	MG	5	4	113.31	credit_card
4	2017	9	BA	health_beauty	SP	17	5	343.71	credit_card
5	2017	11	MT	telephony	SP	34	3	29.68	credit_card
6	2017	3	RJ	bed_bath_table	MG	9	5	284.71	credit_card
7	2018	4	SP	computers_acc...	SP	3	5	23.28	credit_card
8	2018	2	SP	electronics	SP	7	4	16.78	credit_card
9	2018	8	PR	furniture_decor	SP	11	5	150.09	credit_card

Figure 6.2.2 : Evidence of the Data Mart 1 creation

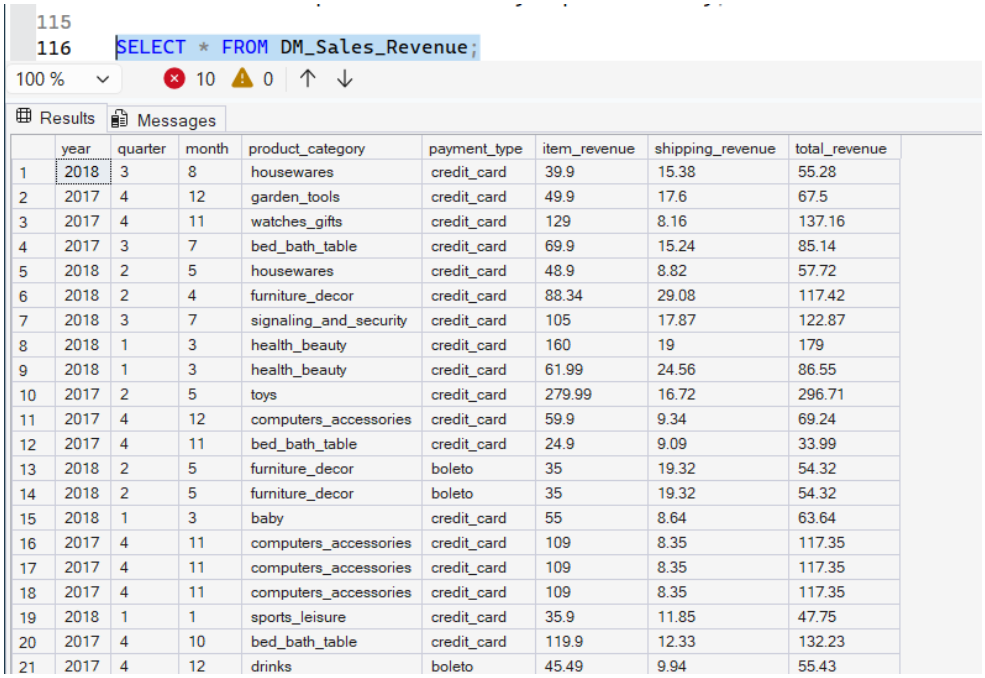
6.3 Department-Specific Data Mart 2: Sales and Revenue Performance

- **Purpose of the Data Mart:** This particular Data Mart looks into the cash tracking and sales performance with the desired aim of getting a faster version of the original data to look at revenue generation, shipping costs, and how customers pay over time.
- **Target Users:** Finance Department, Sales Strategy Teams, and Executive Management.
- **Analytical Benefits:**
  - ✓ It can monitor the revenue at different timestamps such as years, quarters or months (DM_Sales_Revenue).
  - ✓ It highlights the product category revenue in order to find out what product brings the most revenue.
  - ✓ It checks the customers' payment habits (such as credit cards vs boleto) and helps with cash forecast and optimization.

SQL Implementation Script:

```sql
-- Creating the Sales and Revenue Data Mart
CREATE VIEW DM_Sales_Revenue AS
SELECT
    d.year,
    d.quarter,
    d.month,
    p.product_category_name_english AS product_category,
    f.payment_type,
    f.price AS item_revenue,
    f.freight_value AS shipping_revenue,
    f.total_order_value AS total_revenue
FROM Fact_Sales f
JOIN Dim_Date d ON f.Date_Key = d.Date_Key
JOIN Dim_Product p ON f.Product_Key = p.Product_Key;
```

Figure 6.3.1 : SQL Script of Data Mart 2



```sql
SELECT * FROM DM_Sales_Revenue;
```

100 % 10 0

Results Messages

	year	quarter	month	product_category	payment_type	item_revenue	shipping_revenue	total_revenue
1	2018	3	8	housewares	credit_card	39.9	15.38	55.28
2	2017	4	12	garden_tools	credit_card	49.9	17.6	67.5
3	2017	4	11	watches_gifts	credit_card	129	8.16	137.16
4	2017	3	7	bed_bath_table	credit_card	69.9	15.24	85.14
5	2018	2	5	housewares	credit_card	48.9	8.82	57.72
6	2018	2	4	furniture_decor	credit_card	88.34	29.08	117.42
7	2018	3	7	signaling_and_security	credit_card	105	17.87	122.87
8	2018	1	3	health_beauty	credit_card	160	19	179
9	2018	1	3	health_beauty	credit_card	61.99	24.56	86.55
10	2017	2	5	toys	credit_card	279.99	16.72	296.71
11	2017	4	12	computers_accessories	credit_card	59.9	9.34	69.24
12	2017	4	11	bed_bath_table	credit_card	24.9	9.09	33.99
13	2018	2	5	furniture_decor	boleto	35	19.32	54.32
14	2018	2	5	furniture_decor	boleto	35	19.32	54.32
15	2018	1	3	baby	credit_card	55	8.64	63.64
16	2017	4	11	computers_accessories	credit_card	109	8.35	117.35
17	2017	4	11	computers_accessories	credit_card	109	8.35	117.35
18	2017	4	11	computers_accessories	credit_card	109	8.35	117.35
19	2018	1	1	sports_leisure	credit_card	35.9	11.85	47.75
20	2017	4	10	bed_bath_table	credit_card	119.9	12.33	132.23
21	2017	4	12	drinks	boleto	45.49	9.94	55.43

Figure 6.3.2 : Evidence of the Data Mart 2 creation

Task 7: OLAP Analysis and Business Intelligence Dashboard Development

7.1 Visualization Strategy

Microsoft Power BI served an important role in the presentation layer in obtaining actionable business intelligence from the Data Warehouse. Instead of retrieving data directly from the raw operational database or querying the entire Star Schema, Power BI connected through the Import connectivity to the dedicated SQL Server Data Marts (DM_Customer_Experience and DM_Sales_Revenue). This architecture enables fast and optimized querying performance and visualizations rendering and delivers a report-ready clean data model.

7.2 DAX Measures Implementation

To calculate dynamic Key Performance Indicators (KPIs) and aggregation metrics across interactive filters, custom Data Analysis Expressions (DAX) were authored:

1. Total Revenue:

```
Total Revenue = SUM('DM_Sales_Revenue'[total_revenue])
```

2. Average Customer Review Score:

```
Average Review Score = AVERAGE('DM_Customer_Experience'[review_score])
```

3. Average Delivery Duration:

```
Average Delivery Duration =
AVERAGE('DM_Customer_Experience'[delivery_duration_days])
```

7.3 Dashboard Structure and Implementation

To meet the analytical requirements specified in the project guidelines, the Business Intelligence report was structured into three dedicated interactive pages:

Page 1: Executive Summary

- **Objective:** Provides C-level executives with an immediate high-level performance overview of the e-commerce business.
- **Visual Components:**
  - ✓ KPI Summary Cards: Displays core metrics including Total Revenue, Average Review Score, and Average Delivery Duration.
  - ✓ Donut Chart (Revenue by Payment Type): Illustrates the revenue contribution across different payment methods (Credit Card, Boleto, Voucher, Debit Card).
  - ✓ Clustered Bar Chart (Top 5 Revenue Categories): Highlights the top 5 performing product categories by total gross earnings.

Page 2: Trend Analysis

- **Objective:** Focuses on time-series analysis to evaluate revenue patterns, seasonality, and fulfillment performance across different time horizons.
- **Visual Components:**
  - ✓ Line Chart (Revenue Trend Over Time): Implements a Date Hierarchy (Year → Quarter → Month) enabling drill-down capabilities to track sales momentum.
  - ✓ Clustered Column Chart (Quarterly Revenue by Year): Compares quarterly revenue performance year-over-year.
  - ✓ Area Chart (Avg Delivery Days by Month): Tracks monthly delivery fulfillment trends to identify seasonal operational bottlenecks.

Page 3: Interactive Analysis

- **Objective:** Facilitates ad-hoc data exploration, multi-dimensional slicing, and interactive root-cause analysis.
- **Visual Components:**
  - ✓ Interactive Slicer Panel: Features dynamic filters for Order Year, Customer State, and Product Category.
  - ✓ Drill-Down Bar Chart (Revenue by Product Category): Enables hierarchical navigation through product performance.
  - ✓ Scatter Plot (Delivery Time vs Customer Satisfaction): Correlates Average Delivery Duration against Average Review Score across product categories to visualize shipping impact on customer satisfaction.
  - ✓ Treemap Visual (Revenue Distribution by State): Represents geographic sales concentration across states using proportional, heat-mapped blocks.

7.4 Evidence of Interactive Dashboards

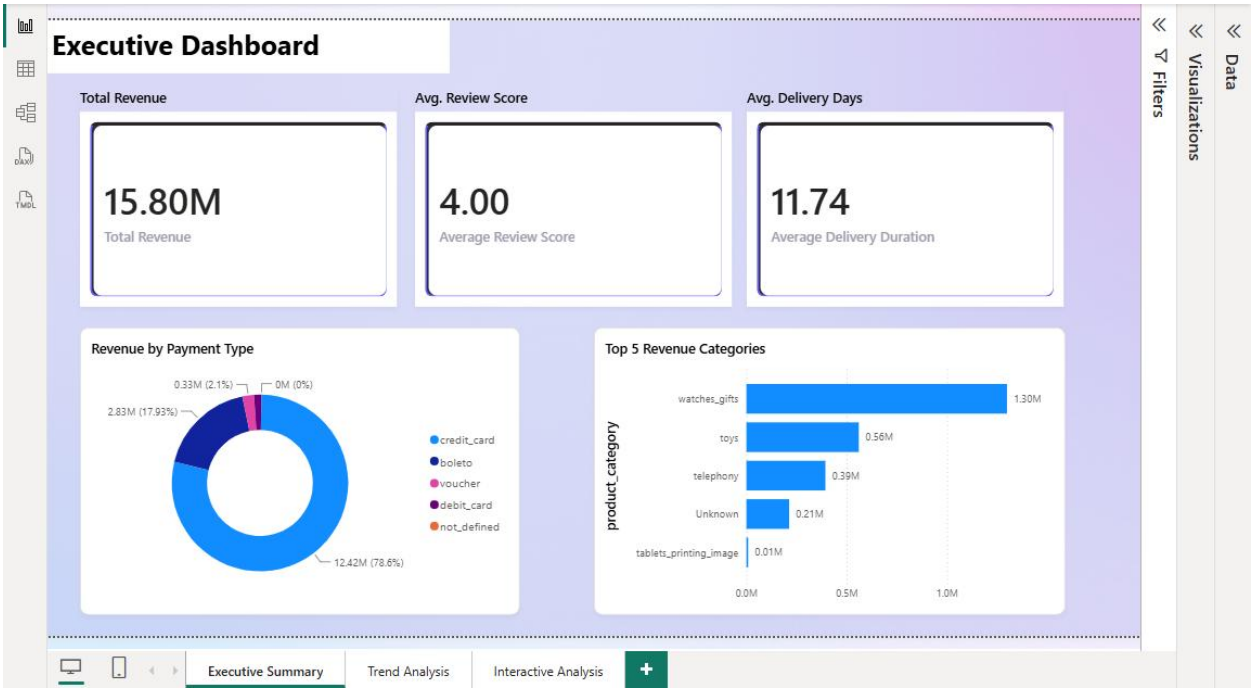


Figure 7.4.1 : Page 1 - Executive Summary Dashboard

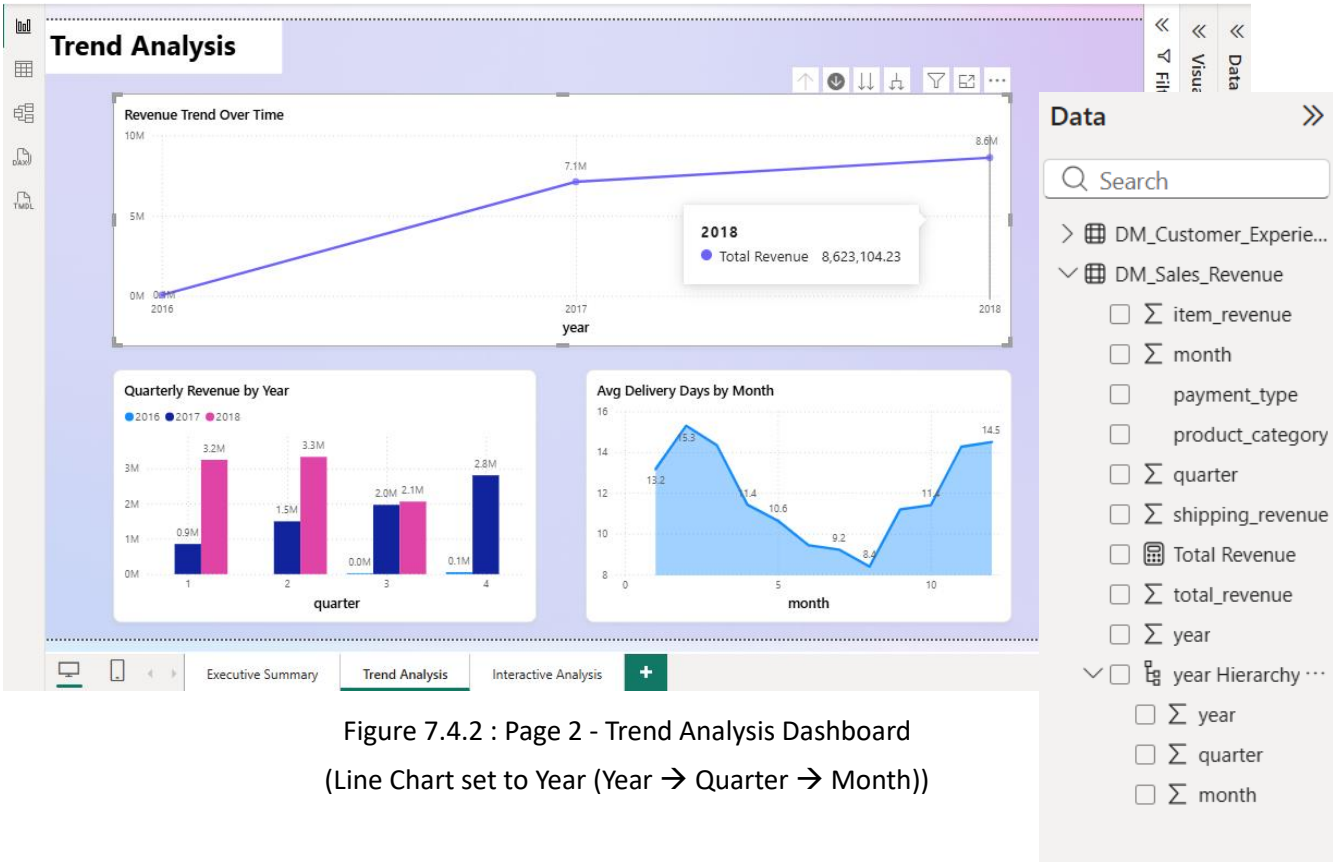


Figure 7.4.2 : Page 2 - Trend Analysis Dashboard

(Line Chart set to Year (Year → Quarter → Month))

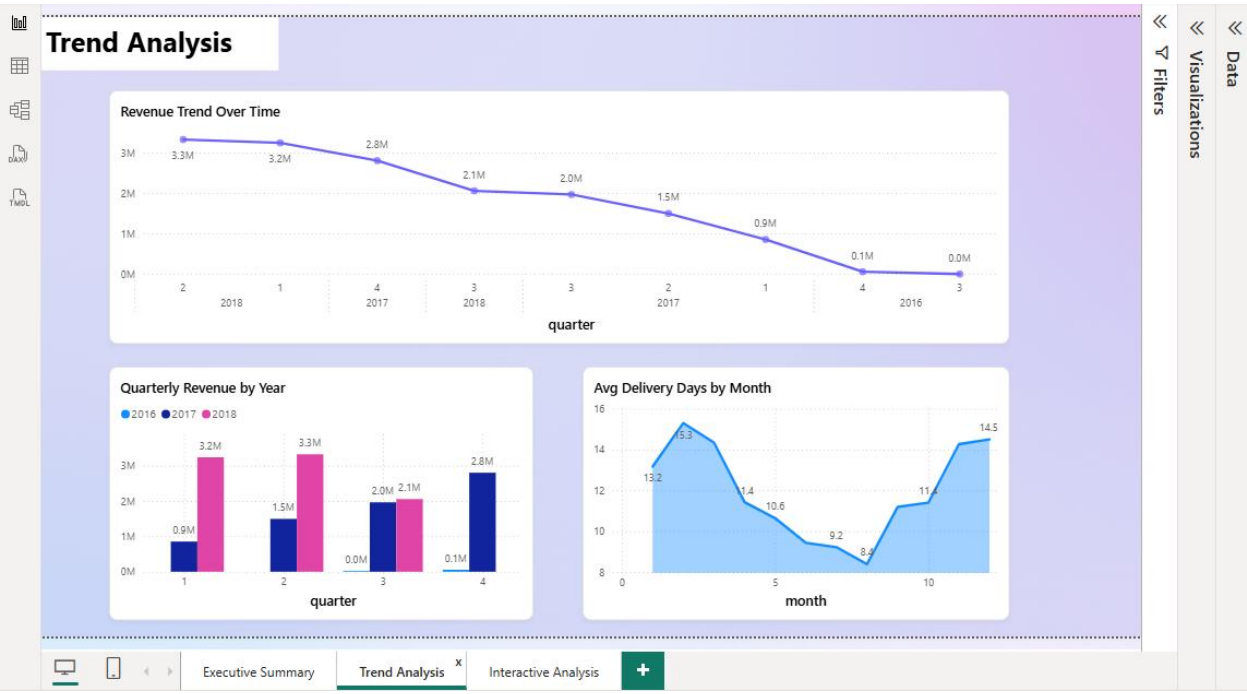


Figure 7.4.3 : Page 2 - Trend Analysis Dashboard

(Line Chart set to quarter (Year → Quarter → Month))

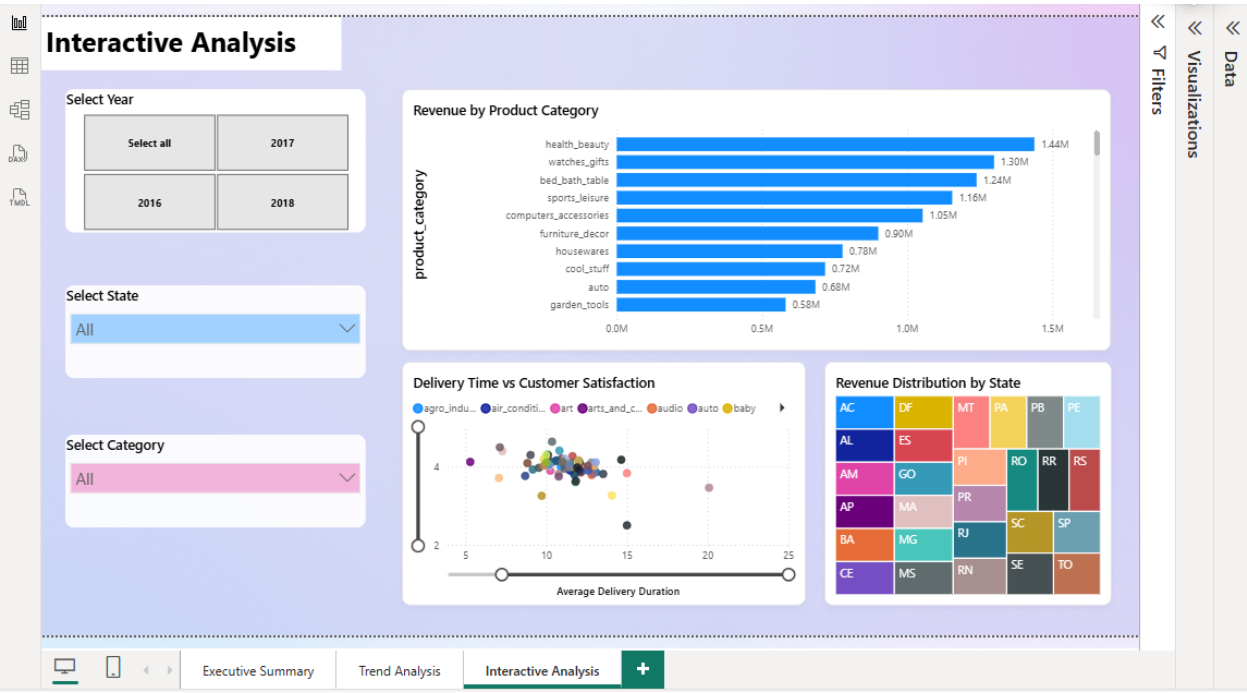


Figure 7.4.4 : Page 3 - Interactive Analysis Dashboard

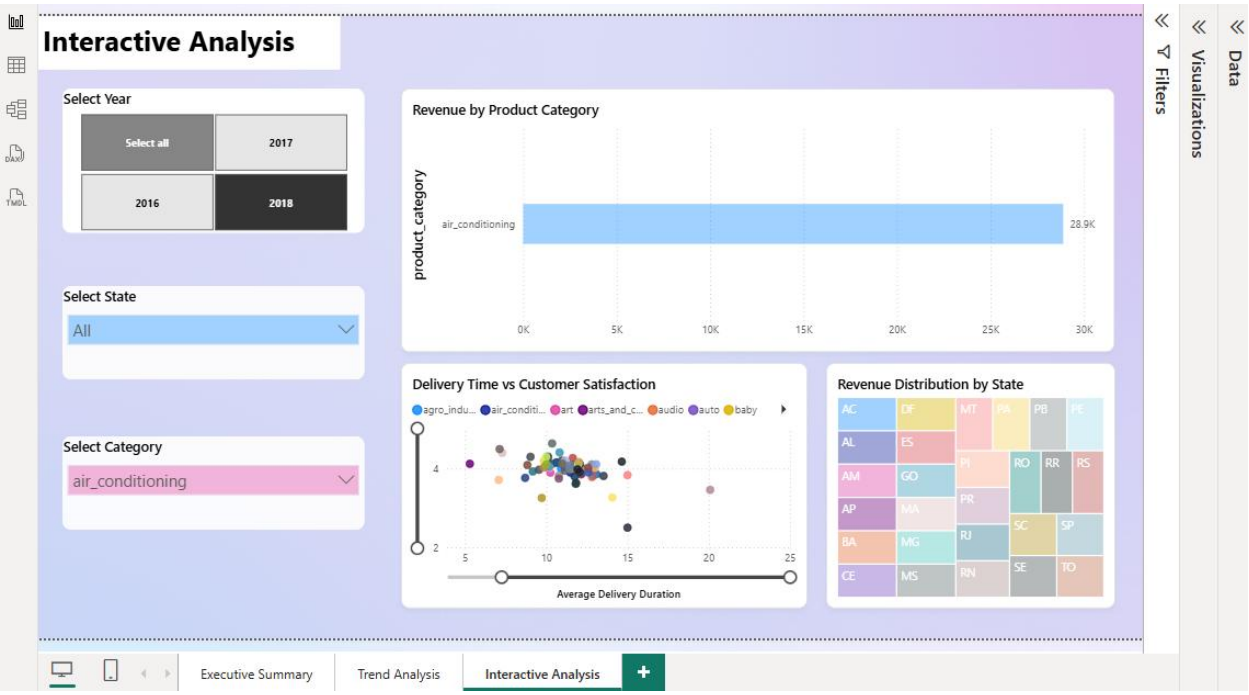


Figure 7.4.5 : Page 3 - Interactive Analysis Dashboard

(Year 2018 and Product Category set to 'air_conditioning')

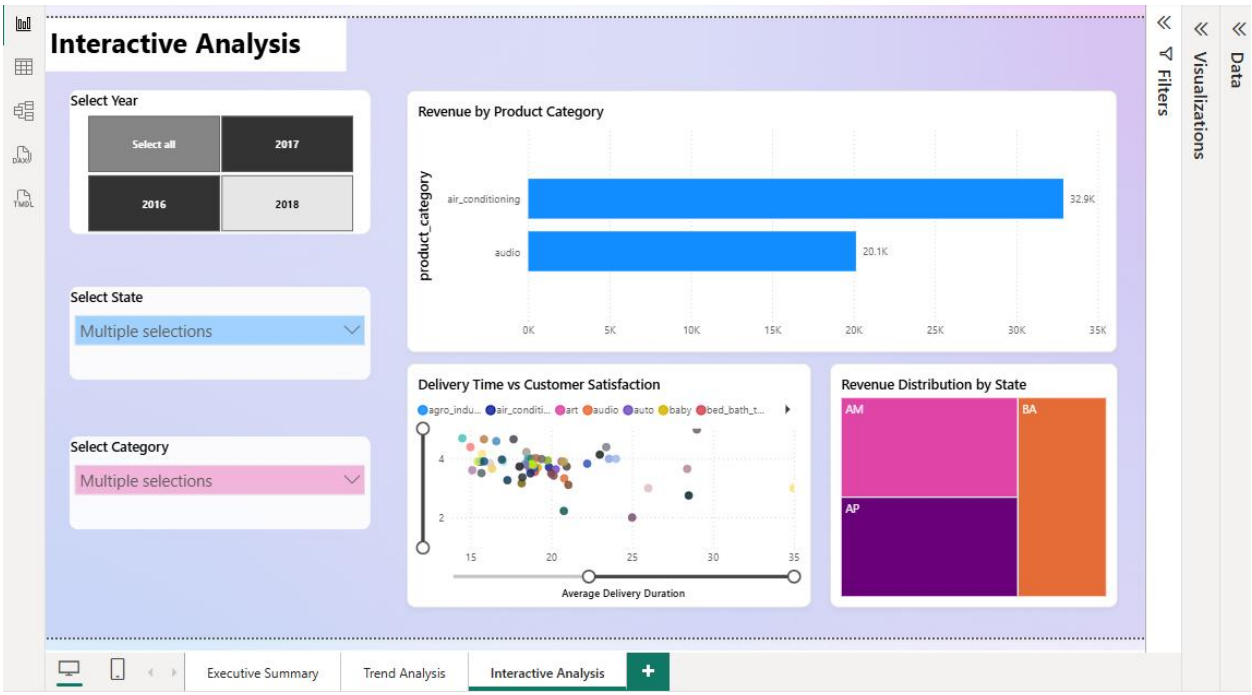


Figure 7.4.6 : Page 3 - Interactive Analysis Dashboard

(Year 2016,2017 and Product Category set to 'air_conditioning', ‘audio’ and State set to ‘AM’, ‘AP’, ‘BA’)

Task 8: Business Insights and Recommendations

1. Operational Performance: Impact of Delivery Duration on Customer Reviews

- **Insight:** The guide that analyses delivery_duration_days along with review_score shows that there is a significant decrease in customer satisfaction when delivery is prolonged beyond a certain threshold. Orders that are delivered on time, within 3–5 days, usually score 4 or 5 stars, while those that are delivered late, after 10–14 days, often receive 1- or 2-star ratings because of delays in shipping.
- **Decision-Making Support:** Creating a map of dissatisfied customers in order to evaluate courier companies and eliminate those incapable of meeting delivery SLAs.
- **Recommendation:** Introduce notifications about delayed delivery and punish companies breaching deadlines.

2. Customer Payment Preferences: Dominance of Electronic and Installment Methods

- **Insight:** The distribution of payment types found that Credit Cards (generally paid in installments) and Boleto (bank slips) have accounted for the highest revenues, while debit cards and vouchers share a smaller portion.
- **Decision-Making Support:** Advises the accounting department on cash flow budgeting and the improvement of payment gateways.
- **Recommendation:** Keep an automatic tie-up with the leading credit card processors and utilize incentives for the use of prepaid accounts to minimize payment reclamation issues.

3. Sales Trends and Seasonality: Fiscal Fluctuations and Peak Periods

- **Insight:** The time series data has shown the peculiarities of seasonality – revenue peaks (in certain holiday months) have alternate with predictable slumps.
- **Decision-Making Support:** Helps inventory planners and warehouse managers to avoid stock-outs during high-demands months and to minimize holding costs during low season.
- **Recommendation:** Launch focused promotions and discount packages in low earning months.

4. Regional Disparities: Geographic Concentration of Order Volume

- **Insight:** Processing data by customer_state shows that a big part of total orders as well as earnings happens in southern metropolitan areas while distant states experience higher transportation expenses and longer delivery times.
- **Decision-Making Support:** Determines regional development plans, as well as locations for additional distribution centers.
- **Recommendation:** Work with local fulfillment companies around smaller cities.

5. Product Category Performance: High-Revenue Drivers vs. Volume Products

- **Insight:** Analysis of product categories shows that high-revenue groups (for instance, health-and-beauty products, watches, and computers) are the biggest contributors to profits, while high-volume products do not provide adequate profit considering delivery costs incurred by the company.
- **Decision-Making Support:** Helps marketing and merchandise departments to focus their promotions on products providing higher profits.
- **Recommendation:** Give priority to the selling interface on the e-commerce platform to top-hit products while negotiating lower shipment prices for heavy goods with the vendors.

6. Seller Ecosystem: Inter-State Shipping Efficiency and Bottlenecks

- **Insight:** Comparing seller_state to customer_state reveals that cross-state deliveries (where both buyers and sellers exist in different states) take longer than ordinary deliveries and cost a lot more.
- **Decision-Making Support:** Makes it easier to manage seller registration and inventory placement near popular consumer markets.
- **Recommendation:** Tell sellers to keep products that are frequently ordered by consumers in regional warehousing facilities to minimize delays in shipping products across countries.

7. Freight Cost Impact: The Ratio of Shipping Cost to Item Price

- **Insight:** The analysis of freight_value to the actual price data reveals that for cheaper products, freight costs are surprisingly high, which should often correlate with lower customer ratings.
- **Decision-Making Support:** Provides information on pricing and subsidies as well as shipping costs transparency for customers.
- **Recommendation:** Set the thresholds for free delivery and introduce subsidies whereby customers purchasing boxes larger than a set amount of money get free delivery.

9. AI Acknowledgement

As per the guidelines for evaluation, the AI tools were quite helpful in creating the assignments, as it helped to generate ideas for structuring the content, debugging the SQL code, and modifying the business interpretation. Every output provided was evaluated critically for ensuring its accuracy and compliance with the actual Olist dataset behavior.